For loops and range

Loops over lists, strings and ranges, the same task with `while` and `for`, running totals, and positions.

THE QUESTION

How do you run the same steps once for each item of a list, or for each number from 1 to 100?

THE INTUITION

A `for` loop gives its loop variable each item of a sequence in turn and runs the body once per item. `range` makes a sequence of numbers.

KEY TAKEAWAY

`range` stops just before stop. Use `for` when you know what to walk through, `while` when you only know when to stop.

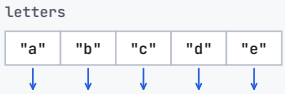
`for x in items:` → `x = the next item` → `run the body` → `items left?` → `yes: back to the for line` → `no: carry on after the loop`

1 `for` gives the loop variable each item in turn

a list: one item per pass

```
letters = ["a", "b", "c", "d", "e"]
for x in letters:
    print(x, end=" ")
print("done")
```

prints `a b c d e done`



each pass: x is the next item

a string: one character per pass

```
word = "data"
count = 0
for ch in word:
    if ch == "a":
        count = count + 1
print(count, ch)
```

prints `2 a`

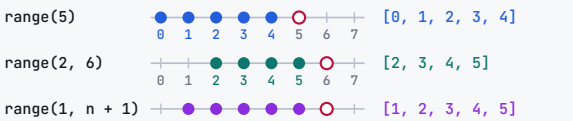
pass	ch	ch == "a"	count after
1	"d"	False	0
2	"a"	True	1
3	"t"	False	1
4	"a"	True	2

after the loop ch is still "a", the last character

A **for loop** runs its body once for each item of a **sequence**: a list, a string or a range. Before each pass the `for` line puts the next item in the **loop variable** (`x`, `ch`); this walk is called **iterating**. There is no index, test or update to write.

2 `range` counts up to stop, and leaves stop out

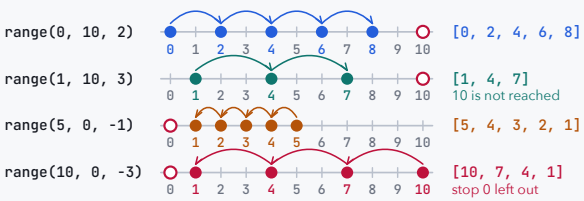
dot: a number the range gives · open circle: stop, never given




```
range(5)          → range(0, 5)      sum(range(1, n + 1)) → 15
len(range(2, 6))  → 4                range(10)[3]         → 3
```

`range(stop)` counts from 0 to stop - 1: `range(5)` gives five numbers, the indexes of a 5-item list. `range(start, stop)` begins at start, so 1 to n is `range(1, n + 1)`. A **range** makes its numbers when asked, so printing one shows `range(0, 5)`; `list()` shows the numbers.

3 A step jumps; a negative step counts down




```
list(range(5, 2)) → []   list(range(2, 5, -1)) → []   list(range(0)) → []
list(range(0, 10, 0)) → ValueError: range() arg 3 must not be zero
```

The third number, the **step**, is the jump. Counting stops before reaching or passing stop, and a negative step counts down to a stop that is still left out. If start is already past stop, the range is **empty** and a loop over it runs zero times. The notebook's odd numbers, `range(1, 11, 2)`, add up to 25.

4 The same task with `while` and with `for`

while: you write all three parts

```
n = 5
s = 0
i = 1                  start
while i <= n:          test
    s = s + i
    i = i + 1          update
print(s)
```

prints `15`

for: range(start, stop, step)

```
n = 5
s = 0
for i in range(1, n + 1):
    s = s + i
print(s)
```

prints `15`

stop is n + 1: the test was i <= n
no step given: it is 1

```
n = 3                  start
while n > 0:           test
    print(n, end=" ")
    n = n - 1          update
print("Liftoff!")
```

prints `3 2 1 Liftoff!`

```
for n in range(3, 0, -1):
    print(n, end=" ")
print("Liftoff!")
```

prints `3 2 1 Liftoff!`

the same output from three lines

`range(...)` holds the start, test and update that a `while` loop spreads over three lines. Use `for` when you know what to walk through, `while` when the loop stops on a condition you cannot count ahead, such as reading input until 0 (see the while loops sheet).

5 Total, count, new list and best in one loop

scores = [90, 82, 83, 100, 102]

```
total = 0
count = 0
plus4 = []
best = scores[0]
for x in scores:
    total = total + x
    if x > 95:
        count = count + 1
    plus4.append(x + 4)
    if x > best:
        best = x
print(total, count, best)
print(plus4)
```

x	total	count	best	appends
90	90	0	90	94
82	172	0	90	86
83	255	0	90	87
100	355	1	100	104
102	457	2	102	106

start: total 0, count 0, plus4 [], best 90

prints `457 2 102`

`[94, 86, 87, 104, 106]`

the list plus4 after the loop

The lists sheet's patterns without an index. Set each result before the loop: total and count at 0, a new list at `[]`, the best at the first item. `sum` and `max` agree: 457 102. The notebook's factorial over `range(1, n + 1)` gives 3628800 for n = 10.

6 Positions: `range(len(x))` or `enumerate`

tasks = ["eat", "study", "tennis"]

index by index

```
for i in range(len(tasks)):
    print(i + 1, tasks[i])
```

enumerate: index and item together

```
for i, t in enumerate(tasks, start=1):
    print(i, t)
```

pass	i	tasks[i]	i, t
1	0	"eat"	1, "eat"
2	1	"study"	2, "study"
3	2	"tennis"	3, "tennis"

both print
```
1 eat
2 study
3 tennis
```

```
list(enumerate(tasks)) → [(0, 'eat'), (1, 'study'), (2, 'tennis')]
```

For position and item, loop over `range(len(tasks))`, the indexes `[0, 1, 2]`, and read `tasks[i]`. **enumerate** hands out (index, item) pairs, a kind of **tuple**; the `for` line **unpacks** each into two names, and `start=1` counts from 1. `zip`, for two lists side by side, is on the iterables sheet.

What a loop can and cannot change

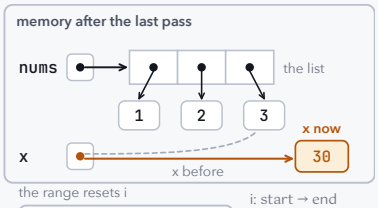
Assigning to the loop variable, removing items mid-loop, `break` and `continue` in a `for`, and a loop inside a loop.

7 Assigning to x does not change the list

loop over the items

```
1 nums = [1, 2, 3]
2 for x in nums:
3     x = x * 10
4 print(nums, x)
```

prints `[1, 2, 3] 30`



write through the index instead

```
1 nums = [1, 2, 3]
2 for i in range(len(nums)):
3     nums[i] = nums[i] * 10
4 print(nums)
```

prints `[10, 20, 30]`

the range resets i

```
1 for i in range(3):
2     print(i, end=" ")
3     i = i + 10
```

prints `0 1 2`

i: start → end

```
0 → 10
1 → 11
2 → 12
```

x names the current item. `x = x * 10` makes a new value and moves x to it; the list keeps its items. To change the list, assign to `nums[i]`. With a range, the next pass takes the next number whatever `i` became (see the Python memory sheet).

8 Removing items mid-loop skips some

remove the even numbers

```
1 nums = [2, 4, 6, 7]
2 for x in nums:
3     if x % 2 == 0:
4         nums.remove(x)
5 print(nums)
```

prints `[4, 7]`

2 and 6 went; 4 stayed

the loop keeps a hidden position 0, 1, 2, ...

pass 1 position 0	2	4	6	7	x = 2: removed, others slide left
pass 2 position 1	4	6	7		x = 6: removed, 4 is skipped
then position 2	4	7			past the end: the loop stops

fix: loop over a copy

```
1 for x in nums[:]:
2     if x % 2 == 0:
3         nums.remove(x)
4 print(nums)
```

prints `[7]`

or build a new list

```
1 odds = []
2 for x in nums:
3     if x % 2 != 0:
4         odds.append(x)
5 print(odds)
```

prints `[7]`

The loop keeps a hidden position that goes up by 1 each pass. Removing the current item slides the rest left, so the next item lands on the used position and is never seen: 2 passes, 4 items. Loop over a copy, `nums[:]` (the slicing sheet), or build a new list.

9 `break` and `continue` work in a `for` too

scores = [90, 82, 83, 100, 102]

```
1 for x in scores:
2     if x > 95:
3         print("found", x)
4         break
5 print("after the loop")
```

x > 95 ?

90	82	83	100	~~102~~
no	no	no	break	not seen

prints `found 100` `after the loop`

the notebook: add up the odd numbers

```
1 total = 0
2 for i in range(1, 11):
3     if i % 2 == 0:
4         continue
5     total = total + i
6 print(total)
```

i from range(1, 11)

1	2	3	4	5	6	7	8	9	10

green: added · grey: continue skipped it

`1 + 3 + 5 + 7 + 9 = 25`

prints `25`

`break` leaves the loop at once: 102 is never looked at. `continue` ends this pass; unlike in a while loop, no update can be skipped. `range(1, 11, 2)` gives 25 with no `if`. More on the break, continue and nested loops sheet.

10 A `for` inside a `for`: the inner one runs in full

a times table

```
1 for i in range(1, 4):
2     for j in range(1, 5):
3         print(i * j, end=" ")
4     print()
```

inner loop restarts for every i: 3 × 4 = 12 prints

i \ j (inner loop)	1	2	3	4
1	1	2	3	4
2	2	4	6	8
3	3	6	9	12

each row is one pass of i; print() ends the line

rows = [[1, 2], [3], [4, 5, 6]]

```
1 for row in rows:
2     for v in row:
3         flat.append(v)
```

flat = [] before the loop

prints `[1, 2, 3, 4, 5, 6]`

the inner loop runs 2, 1, then 3 times

For each `i`, the inner loop starts again at `j = 1` and runs all its passes. It can also walk what the outer loop hands it, such as each `row` of a list of lists.

11 Common mistakes: what you wrote, what Python says or prints, the fix

you wrote	the last line Python prints, or the wrong output; why, and the fix
`for x in 5:` `  print(x)`	`TypeError: 'int' object is not iterable` a number is not a sequence; `for x in range(5):` prints 0 1 2 3 4
`for i in range(1, 10):` `  total = total + i`	`prints 45` stop is left out, so 10 is never added; `range(1, 11)` prints 55
`for i in range(len(nums)):` `  total = total + i`	`prints 3` this adds the indexes 0, 1, 2 of `nums = [5, 10, 20]`; add `nums[i]` and it prints 35
`for t in range(0, 1, 0.1):`	`TypeError: 'float' object cannot be interpreted as an integer` range takes whole numbers only: loop k over `range(10)` and use `t = k / 10`
`evens = range(0, 10, 2)` `evens + [10]`	`TypeError: unsupported operand type(s) for +: 'range' and 'list'` a range is not a list: `list(evens) + [10]` gives `[0, 2, 4, 6, 8, 10]`

12 Try it (answers in green)

1. `list(range(3, 12, 4))`?
`[3, 7, 11]`: 15 would pass 12.

2. `range(10, 0, -2)`: which numbers, how many?
`[10, 8, 6, 4, 2]`, 5: 0 is left out.

3. `for ch in "abc": print(ch * 2, end=" ")`?
`aa bb cc`

4. `for i in range(3): i = i + 5,` then `print(i)`?
7: the last pass began at 2.

5. Sum the positives of `[4, -2, 7, -5]` with a `for`.
11: `if x > 0: t = t + x`

6. Inner passes of `for j in range(i)` inside `for i in range(4)`?
6: 0 + 1 + 2 + 3.

one pass per item · stop is left out · for: known items, while: a stop condition · change `x[i]`, not `x`

Source: the for loop notebook (range, for versus while, enumerate, factorial, odd sums), nipunbatra.github.io/comp22. Every output, trace value and error message comes from running the code with Python 3.11.11. Next: the Python memory sheet.